

Project Write-Up

Cost-Aware Terrain Pathfinding with Classical Search Algorithms

CMPT 310 - D100, Introduction to Artificial Intelligence, Summer 2026 | Team Pathfinders: Banveet Johal, Lucas Zhang, Sheng Zhan

1. System Explanation

What does the system do?

The system generates 2D terrain grids with elevation and obstacles, converts each grid into a cost-weighted graph, and finds a route from a start cell to a goal cell using five different classical search algorithms. It then evaluates and visualizes how each algorithm's path compares on cost, length, nodes explored, and runtime, so the trade-offs between uninformed, cost-sensitive, and heuristic-guided search become directly observable.

What AI methods are used, and how?

- **Uninformed search** - Breadth-First Search (FIFO queue) and Depth-First Search (stack), neither of which uses edge cost or a heuristic.
- **Cost-sensitive uninformed search** - Uniform-Cost Search, a priority queue ordered by accumulated path cost; guaranteed to find the minimum-cost path.
- **Informed (heuristic) search** - Greedy Best-First Search, ordered purely by a Manhattan-distance heuristic to the goal, and A*, ordered by cost-so-far plus that same heuristic, combining optimality with goal-directed efficiency.

AI pipeline

Data flows through five stages, each implemented as an independent, testable module under `terrain_navigation/`:



Figure 1. Data input (terrain generation) → preprocessing (weighted graph construction) → model (five search algorithms) → output (metrics + visualization).

Limitations

- **Grid-only movement.** The graph only connects 4-directional neighbours, so paths cannot move diagonally even when that would be shorter or cheaper. *Mitigation:* this is a deliberate scope decision documented in `graph.py`; diagonal (8-directional) edges could be added by extending the `directions` list with a corrected diagonal cost.
- **Greedy and A* rely on a fixed Manhattan-distance heuristic,** which is admissible for 4-directional grids but does not account for elevation, so A*'s efficiency advantage over UCS shrinks on very steep terrain (`mountain_barrier`). *Understanding:* we confirmed this by comparing nodes-explored across all three scenarios rather than just one.
- **Static, precomputed terrain.** Elevation and obstacles are fixed before search begins; the system does not support dynamic obstacles or replanning. *Mitigation:* out of scope for Milestone 1, called out here rather than silently ignored.

2. Feature Table

Key components of the AI system. All components are fully working, tested against the three scenarios, and integrated into the single-command pipeline run by `main.py`.

Description	Platform	Completeness	Code	Author(s)	Notes
Terrain & obstacle generation for 3 scenarios (flat, mountain barrier, random)	Local	5	Python	Team	NumPy arrays; reproducible via fixed random seeds
Weighted graph construction from terrain + obstacle mask	Local	5	Python	Team	4-connected grid; edge cost = 1 + elevation delta
Uninformed search: BFS, DFS	Local	5	Python	Team	Baselines used to highlight cost-blind search behaviour
Cost-sensitive search: Uniform-Cost Search	Local	5	Python	Team	heapq priority queue; verified optimal on all 3 scenarios
Informed search: Greedy Best-First, A*	Local	5	Python	Team	Manhattan-distance heuristic; A* matches UCS's optimal cost
Path metrics: cost, length, elevation gain, nodes explored, runtime	Local	5	Python	Team	Recomputed independently of the search algorithm for a fair comparison
Visualization: per-algorithm path overlay + comparison bar charts	Local	5	Python	Team	Matplotlib; auto-saved to <code>outputs/<scenario>/</code>
Poster & submission document generation	Local	5	Python	Team	Matplotlib/ReportLab scripts under <code>poster/</code> and <code>docs/</code>

Completeness scale: 5 = fully working, clean, tested and integrated; 4 = working but missing polish; 3 = partially complete; 2 = proof of concept; 1 = research/prototype only; 0 = not implemented. **Author(s)** is listed as "Team"

because all components were developed collaboratively; see the group's internal commit history for individual contribution details.

3. External Tools & Libraries

- **NumPy** - 2D elevation arrays and boolean obstacle masks for all three terrain scenarios (`terrain.py`).
- **Matplotlib** - all path/comparison visualizations (`visualize.py`) and the poster/document figures.
- **Python standard library** - `heapq` for the UCS/Greedy/A* priority queues and `collections.deque` for the BFS queue.

No external datasets, pretrained models, or third-party APIs were used - all terrain is procedurally generated. No open-source search-algorithm code was reused; BFS, DFS, UCS, Greedy Best-First, and A* were implemented from scratch in `search.py`. Platform dependencies: Python 3.9+, with exact package versions pinned in `requirements.txt` at the repository root. No installation steps beyond `pip install -r requirements.txt` are required.